Fuzzing the irrigation scheduler

Baseline behavior and failing properties

Objective

Use Hypothesis to find input-validation bugs in the assigned irrigation scheduler.
Source: SETAPESU26/L6_L7_37_irrigation_scheduler, commit 1e1c093.
Environment: Python 3.11, pytest 8.3.4, Hypothesis 6.112.0.

1. Original application

Inputs: north:120, then 40,,50. The program stopped before asking for rainfall.
Captured terminal output, with traceback paths shortened.

```
> python irrigation_scheduler.py
==== Running parse_zone ====
Enter zone:area in m2 (e.g. north:120): Zone and Area: ('North', 120.0)

==== Running moisture_average ====
Enter moisture readings as CSV (e.g. 40,35,50):
Traceback (most recent call last):
  File "irrigation_scheduler.py", line 6, in moisture_average
    values = [int(r) for r in readings_csv.split(',')]
ValueError: invalid literal for int() with base 10: ''
```

2. Fuzz tests against the original

The same 12 tests were run before and after the fix. The excerpt shows the three no-crash failures and the overall result. Six other input-validation tests also failed.

```
> python -m pytest -p _hypothesis_pytestplugin -q --tb=short test_irrigation_scheduler.py
FFF...FFFFFF                                                             [100%]
========================== short test summary info ==========================
FAILED test_irrigation_scheduler.py::test_parse_zone_no_crash
FAILED test_irrigation_scheduler.py::test_moisture_average_no_crash
FAILED test_irrigation_scheduler.py::test_water_needed_no_crash
9 failed, 3 passed in 32.43s
```

What the failures exposed

parse_zone: an empty string cannot be unpacked into a zone and an area.
moisture_average: an empty CSV field cannot be converted to an integer.
water_needed: None raises AttributeError when the function calls split.

Validation and retest

Fixed application behavior and passing properties

Changes to the three functions

parse_zone requires a nonblank zone and a finite, nonnegative area.
moisture_average requires comma-separated integer readings between 0 and 100.
The range assumes the readings are percentages. The result still rounds down.
water_needed checks finite, nonnegative inputs and rejects arithmetic overflow.
Invalid values return None. The CLI prints Invalid input and continues.

3. Fixed application

Inputs: north:120, 40,,50, then 120|8. The invalid CSV is handled and the
water estimate is printed. A separate run with 40,35,50 returned an average of 41.

```
> python irrigation_scheduler.py
==== Running parse_zone ====
Enter zone:area in m2 (e.g. north:120): Zone and Area: ('North', 120.0)

==== Running moisture_average ====
Enter moisture readings as CSV (e.g. 40,35,50): Average Moisture: Invalid input

==== Running water_needed ====
Enter area|rainfall (e.g. 120|8): Water Needed: 592.0
```

4. Fuzz tests against the fixed code

The suite covers arbitrary inputs, generated valid values and targeted edge cases.
All 12 tests passed with the same properties used for the baseline run.

```
> python -m pytest -p _hypothesis_pytestplugin -q --tb=short test_irrigation_scheduler.py
............                                                             [100%]
12 passed in 2.15s
```

Conclusion

The three functions now handle malformed inputs without crashing in the tested
cases. Valid inputs still produce the zone, floor average and water estimate.